

OPEN STUDIO: 12-WEEK / 3-PHASE AGENTIC EXECUTION BLUEPRINT

Machine-Actionable Engineering Roadmap for Antigravity One-Shot Implementation

Target Stack: Tauri v2 (Rust) + React 18/19 (TS) + Monaco	Hardware: Universal Acceleration (NVIDIA, AMD, Apple, Intel, CPU)
Inference: Ollama (SSE) + llama.cpp + MLX	Latency Targets: < 40ms Tab Ghost-Text • < 2s In-line Diffs
Structure: Phase 1 MVP (W1-4), Phase 2 Smart (W5-8), Phase 3 Ready (W9-12)	Air-Gap Guarantee: Verifiable Zero-Telemetry CI Verified

PHASE 1 (WEEK 1): DESKTOP SHELL & EDITOR INFRASTRUCTURE

DAY 1: PROJECT SCAFFOLDING & TAURI V2 SHELL

Objective:	Setup production Tauri v2 workspace with Rust core, Vite + React + TS, and Tailwind CSS in an air-gapped dark theme.
Deliverables:	src-tauri/Cargo.toml, tauri.conf.json (offline CSP), AppSystemInfo IPC bridge (get_system_info).
Interface / Contract:	get_system_info() -> Result returning OS, architecture, total RAM.
Antigravity Prompt:	"Scaffold Tauri v2 + React 18 + TS + Tailwind. Implement get_system_info IPC command. Configure strict offline CSP. Verify cargo check & npm run build."
Test Command:	cd src-tauri && cargo check && cd .. && npm run build

DAY 2: MONACO EDITOR CORE & OFFLINE BUNDLING

Objective:	Integrate Monaco Editor with local worker bundling via Vite, multi-tab buffer store, split panes, and dirty tracking.
Deliverables:	MonacoEditor.tsx (open-studio-dark theme), editorStore.ts (Zustand), vite.config.ts worker setup, TabBar.tsx.
Interface / Contract:	interface EditorBuffer { id: string; filePath: string; content: string; isDirty: boolean; }
Antigravity Prompt:	"Bundle Monaco locally without CDNs. Implement Zustand editorStore for open buffers, tabs, and split panes. Wire Ctrl+S, Ctrl+W, Ctrl+\.."
Test Command:	npm run test -- -t 'editorStore' && npm run build

DAY 3: NATIVE FILE EXPLORER & VIRTUALIZED TREE

Objective:	Build native filesystem watcher and recursive directory explorer capable of managing 50,000+ files without UI lag.
Deliverables:	src-tauri/src/fs/ (notify-debouncer-mini, walkdir), FileTree.tsx (virtualized list, git status badges).
Interface / Contract:	read_workspace_tree(root: String, depth: usize) -> Result; emits 'workspace-fs-change'.
Antigravity Prompt:	"Implement read_workspace_tree respecting .gitignore. Stream fs-change events via notify. Render virtualized tree linking to editorStore."
Test Command:	cd src-tauri && cargo test fs_tree && cd .. && npm run test -- -t 'FileTree'

DAY 4: EMBEDDED TERMINAL INFRASTRUCTURE (RUST PTY + XTERM.JS)

Objective:	Embed native cross-platform PTY terminal supporting PowerShell/cmd and bash/zsh with ANSI color streams and resize handling.
Deliverables:	src-tauri/src/terminal/ (portable-pty), TerminalPanel.tsx (xterm.js + fit addon).
Interface / Contract:	spawn_terminal_session(id, shell), write_terminal_input(id, data), resize_terminal(id, cols, rows).
Antigravity Prompt:	"Build PTY backend with portable-pty crate. Stream terminal bytes to frontend xterm.js. Wire resize handler and input dispatch."
Test Command:	cd src-tauri && cargo test terminal_pty && cd .. && npm run build

DAY 5: WEEK 1 INTEGRATION & TESTING

Objective:	Unify File Tree, Monaco Editor, TabBar, and Terminal Panel into a cohesive workspace shell; run test suite.
Deliverables:	src/App.tsx cohesive layout with draggable panels and collapse toggles, test suite, milestone tag v0.1.0-w1.
Interface / Contract:	Full app launches offline, editor tabs switch smoothly, terminal executes commands without latency.
Antigravity Prompt:	<i>“Perform Week 1 integration pass. Connect FileTree, Monaco, Terminal in App.tsx. Run end-to-end typing and terminal execution tests.”</i>
Test Command:	<code>npm run build && cd src-tauri && cargo check --release</code>

PHASE 1 (WEEK 2): INFERENCE GATEWAY & INLINE AUTOCOMPLETE

DAY 6: OLLAMA INFERENCE GATEWAY & SSE STREAMING

Objective:	Build multi-backend async HTTP streaming client for Ollama with SSE token streaming, abort controls, and health-check ping.
Deliverables:	src-tauri/src/inference/ (request SSE client), stream_completion command emitting 'llm-chunk', abort_completion.
Interface / Contract:	<code>stream_completion(window, request_id, req: CompletionRequest) -> Result<(), String></code> with keep_alive support.
Antigravity Prompt:	<i>"Implement Ollama streaming SSE gateway using request. Emit chunk events keyed by request_id. Implement cancellation via oneshot channels."</i>
Test Command:	<code>cd src-tauri && cargo test inference_gateway</code>

DAY 7: INFERENCE PRIORITY QUEUE & CONCURRENCY SENTINEL

Objective:	Implement request priority queue ensuring latency-critical autocomplete immediately preempts background chat or indexing.
Deliverables:	src-tauri/src/inference/queue.rs with 4 priority levels (Abort > Autocomplete > Chat > Background).
Interface / Contract:	Priority queue yields low-priority tasks when inline completion is requested; prevents Ollama resource contention.
Antigravity Prompt:	<i>"Implement Inference Priority Queue in Rust (src-tauri/src/inference/queue.rs) with 4 priority levels to guarantee sub-40ms autocomplete."</i>
Test Command:	<code>cd src-tauri && cargo test inference::queue</code>

DAY 8: INLINE GHOST-TEXT AUTOCOMPLETE ENGINE

Objective:	Build sub-40ms inline Tab autocompletion inside Monaco powered by resident Qwen2.5-Coder-1.5B.
Deliverables:	inlineProvider.ts (Monaco InlineCompletionsProvider), FIM prompt formatter, 30ms keystroke debouncer.
Interface / Contract:	<code>formatFIMPrompt(prefix, suffix) -> '< fim_prefix >...< fim_suffix >...< fim_middle >';</code> Tab accepts.
Antigravity Prompt:	<i>"Implement Monaco InlineCompletionsProvider. Extract cursor context, format Qwen FIM prompt, debounce 30ms, query Ollama (keep_alive: -1)."</i>
Test Command:	<code>npm run test -- -t 'inlineProvider'</code>

DAY 9: AUTOCOMPLETE BENCHMARK & LATENCY VERIFICATION

Objective:	Validate sub-40ms Time-To-First-Token (TTFT) and memory footprint under heavy typing simulation.
Deliverables:	bench/ttft.py benchmark harness, automated pass/fail latency assertions (< 40ms average).
Interface / Contract:	<code>bench/ttft.py</code> asserts average TTFT < 40ms across 50 requests; memory profiler verifies < 50MB shell.
Antigravity Prompt:	<i>"Create benchmark script bench/ttft.py sending 50 synthetic FIM requests to Ollama and assert average TTFT < 40ms."</i>
Test Command:	<code>python bench/ttft.py</code>

DAY 10: WEEK 2 POLISH & CONNECTION ERROR RECOVERY

Objective:	Add status bar AI health indicator, automatic reconnect on Ollama restart, and error recovery banners.
Deliverables:	Status bar AI widget ('AI Ready' / 'Ollama Offline'), reconnect listener, milestone tag v0.1.0-w2.
Interface / Contract:	Status bar reflects real-time Ollama connectivity; automatically polls and resumes when Ollama restarts.
Antigravity Prompt:	<i>"Add status bar AI health indicator and automatic reconnect logic for Ollama in src/components/statusbar/."</i>
Test Command:	<code>npm run build && cd src-tauri && cargo check --release</code>

PHASE 1 (WEEK 3): CHAT & DIFF ENGINE

DAY 11: AI CHAT ASSISTANT CORE

Objective:	Build primary AI Chat sidebar with streaming Markdown rendering, code blocks, and conversation state management.
Deliverables:	src/components/chat/ChatPanel.tsx, virtualized message history, markdown renderer with copy buttons.
Interface / Contract:	Chat panel streams tokens in real-time, formats markdown code blocks, supports cancel and retry.
Antigravity Prompt:	<i>"Build AI Chat Assistant in src/components/chat/ChatPanel.tsx with streaming SSE display, markdown highlighting, and Zustand history."</i>
Test Command:	<code>npm run test -- -t 'ChatPanel'</code>

DAY 12: @FILE CONTEXT INJECTION

Objective:	Implement explicit file context injection (@file:path/to/file) allowing users to query specific files without complex RAG.
Deliverables:	@ mention auto-complete dropdown in chat input, file content resolution into prompt context.
Interface / Contract:	Typing '@' presents fuzzy file list; selected file is read from disk and prepended to prompt context.
Antigravity Prompt:	<i>"Implement @file context injection in chat panel. Show fuzzy file dropdown on '@', resolve content, and inject into system prompt."</i>
Test Command:	<code>npm run test -- -t 'fileMention'</code>

DAY 13: FRUGAL SEARCH/REPLACE DIFF PARSER WITH LINE ANCHORS

Objective:	Build deterministic diff parser saving 98% output tokens by generating only modified lines with line-number hints.
Deliverables:	src-tauri/src/diff/frugal_parser.rs parsing SEARCH (line 42) / ===== / REPLACE blocks.
Interface / Contract:	<code>parse_frugal_diff(output) -> Vec</code> ; 3-tier matching (exact, line-anchored, whitespace-trimmed).
Antigravity Prompt:	<i>"Implement Frugal Diff Parser in Rust with line-number hints (SEARCH (line 42)). Implement exact, anchored, and trimmed matching."</i>
Test Command:	<code>cd src-tauri && cargo test diff::frugal_parser</code>

DAY 14: MONACO MULTI-FILE DIFF REVIEW UI

Objective:	Build interactive diff review interface in Monaco supporting inline annotations and side-by-side reviews.
Deliverables:	src/components/diff/DiffReviewModal.tsx, Monaco deltaDecorations for hunks, Accept/Reject keyboard actions.
Interface / Contract:	<code>interface FileDiffState { filePath: string; hunks: Array<{ id: string; status: 'pending' 'accepted' 'rejected' }> }</code>
Antigravity Prompt:	<i>"Create Monaco diff review UI. Add glyph margin accept/reject buttons, file list with +/- line counters, and disk sync."</i>
Test Command:	<code>npm run test -- -t 'DiffReview'</code>

DAY 15: WEEK 3 INTEGRATION & IN-EDITOR CODE INSERTION

Objective:	Connect chat diff responses directly to active Monaco buffer with 1-click 'Apply Changes'.
Deliverables:	'Apply to Editor' CTA button in chat code blocks routing to diff engine, milestone tag v0.1.0-w3.
Interface / Contract:	Clicking 'Apply to Editor' opens diff review with parsed hunks; accepting changes updates disk and buffer.
Antigravity Prompt:	<i>"Connect chat diff responses to Monaco editor with 1-click 'Apply to Editor' routing to diff review engine."</i>
Test Command:	<code>npm run build && cd src-tauri && cargo check --release</code>

PHASE 1 (WEEK 4): SAFETY NET, SHELL POLISH & MVP EXIT GATE

DAY 16: SHADOW GIT CHECKPOINTS (1-CLICK ROLLBACK)

Objective:	Implement automatic background workspace snapshots before any AI diff application using Git stash/commit objects.
Deliverables:	src-tauri/src/git/checkpoint.rs (refs/ai-checkpoints/), create_checkpoint, list_checkpoints, restore_checkpoint.
Interface / Contract:	<code>create_checkpoint(workspace, summary) -> Result; restore_checkpoint reverts working tree safely.</code>
Antigravity Prompt:	<i>"Implement Shadow Git checkpoints in Rust using git2. Commit to refs/ai-checkpoints/ without touching user HEAD. Add 1-click restore."</i>
Test Command:	<code>cd src-tauri && cargo test git::checkpoint</code>

DAY 17: STATE PERSISTENCE ENGINE

Objective:	Ensure workspace state survives application restarts (open tabs, active file, split layout, recent files).
Deliverables:	src/stores/persistence.ts debounced sync to .openstudio/workspace.json; rehydration on app launch.
Interface / Contract:	<code>Persists open tabs, cursor position, active file, split layout; rehydrates seamlessly on startup.</code>
Antigravity Prompt:	<i>"Build State Persistence Engine in src/stores/persistence.ts. Save open tabs, layout, and recent files into .openstudio/workspace.json."</i>
Test Command:	<code>npm run test -- -t 'persistence'</code>

DAY 18: COMMAND PALETTE (CTRL+SHIFT+P)

Objective:	Implement global fuzzy-searchable Command Palette for all editor, AI, and window actions.
Deliverables:	src/components/palette/CommandPalette.tsx, fuzzy matcher, action registry (open file, toggle terminal, clear chat).
Interface / Contract:	<code>Ctrl+Shift+P opens fuzzy searchable action modal with keyboard navigation (Up/Down/Enter/Esc).</code>
Antigravity Prompt:	<i>"Implement Command Palette in src/components/palette/CommandPalette.tsx with fuzzy search over commands, files, and AI actions."</i>
Test Command:	<code>npm run test -- -t 'CommandPalette'</code>

DAY 19: SETTINGS & PREFERENCES UI (CTRL+,)

Objective:	Build clean Settings modal allowing users to configure Ollama URLs, model selections, tab size, and themes.
Deliverables:	src/components/settings/SettingsModal.tsx, .openstudio/settings.json persistence.
Interface / Contract:	<code>Ctrl+, opens Settings modal; persists Ollama endpoint, model selections, theme, and tab preferences.</code>
Antigravity Prompt:	<i>"Build Settings Modal in src/components/settings/SettingsModal.tsx triggered by Ctrl+,. Persist settings to .openstudio/settings.json."</i>
Test Command:	<code>npm run test -- -t 'SettingsModal'</code>

DAY 20: FIRST-RUN ONBOARDING & PHASE 1 MVP EXIT GATE

Objective:	Guide new users on startup: check Ollama health, offer 1-click model pull (Qwen-1.5B), and run full MVP verification.
Deliverables:	src/components/onboarding/OnboardingWizard.tsx, complete Phase 1 test pass, tag v0.1.0-MVP.
Interface / Contract:	<code>Full Phase 1 exit gate: opens repo, provides sub-40ms autocomplete, chat with @file, diffs, and rollback 100% offline.</code>
Antigravity Prompt:	<i>"Build Onboarding Wizard checking Ollama status and offering 1-click model download. Execute full Phase 1 smoke test pass."</i>
Test Command:	<code>npm run build && cd src-tauri && cargo test && cd .. && npm run test</code>

PHASE 2: "IT'S SMART" INTELLIGENCE & CONTEXT (WEEKS 5–8)

Week 5: Codebase Indexing	Tree-sitter AST slicer (800-token skeleton) + BM25 keyword search over raw files for @codebase v1. cargo test ast::slicer
Week 6: Vector RAG & Hybrid	nomic-embed-text + SQLite-vec chunk storage; 2-stage retrieval (Vector top-15 -> BM25 top-3) for full @codebase v2. cargo test rag::hybrid_retrieval
Week 7: LSP & Memory Sentinel	Rust LSP client using lsp-types crate for TS & Python; Monaco diagnostics/hover/def; sysinfo + Ollama /api/ps memory profiler. cargo test lsp::client
Week 8: Router & Auto-Fix	Dynamic task-based model router (Typing->1.5B, Edit->7B, Think->8B) with keep_alive eviction; Terminal regex auto-fix loop. cargo test router::model_router

PHASE 3: "IT'S READY" PRODUCTION & DISTRIBUTION (WEEKS 9–12)

Week 9: Checkpoints & MCP	Full Checkpoint Timeline UI with hover diffs & auto-GC; Native MCP client with schema validation & fallback retry. cargo test mcp::client
Week 10: Plugins & Extension	TypeScript OpenStudioPlugin host; Standalone VS Code extension wedge (FIM autocomplete + frugal diff) for early user acquisition. npm run test:vscode
Week 11: Security & Packaging	Zero-telemetry CI verification (verify-airgap.sh), SQLite encrypted audit log, rules.yaml policy engine, Tauri packaging (.msi, .dmg, .ApplImage). npm run tauri build
Week 12: Launch & Hardening	End-to-end stress testing, production docs (README, SECURITY, PRIVACY), community launch kit (Show HN, r/LocalLLaMA). Tag v1.0.0-GA.